Performance & Platform Admin Guide

This document describes the performance optimizations and admin tooling added in Phase 44D.

---

1. Database Indexes

Production-ready indexes were added to high-traffic tables to speed up the most frequent queries, especially on the Platform Admin and Customer Management portals.

Tables Updated

Table	Indexes Added
`users`	`role`, `active`, `subscriptionTier`, `subscriptionStatus`, `lastSignInAt`, `createdAt`, `currentRestaurantId`
`payments`	composite `[restaurantId, status]`, `[restaurantId, createdAt]`, `[gateway, status]`, `[status, createdAt]`; single `subscriptionId`, `customerEmail`, `processedAt`, `gatewayPaymentId`
`subscriptions`	composite `[restaurantId, status]`; single `tier`, `gateway`, `cancelledAt`, `createdAt`
`notifications`	`restaurantId`, `archived`; composite `[userId, read, archived]`, `[restaurantId, createdAt]`, `[userId, createdAt]`
`restaurants`	`subscriptionTier`, `ownerId`, `trialEndsAt`

All indexes are already present in `prisma/schema.prisma` — Prisma Client has been regenerated and the database is in sync.

Applying to Large Production Databases (CONCURRENTLY)

`prisma db push` creates indexes with a short lock on the table. For very large tables, you may prefer to run these manually using PostgreSQL's `CREATE INDEX CONCURRENTLY` (no lock). Run `scripts/sql/optimize-indexes.sql` with `psql`:

```
psql "$DATABASE_URL" -f scripts/sql/optimize-indexes.sql
```

The script is idempotent (`IF NOT EXISTS`) and safe to re-run.

Monitoring Index Health

Run this query to verify Postgres is using the new indexes:

```
SELECT
  schemaname, relname as table, indexrelname as index,
  idx_scan as scans, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE relname IN ('users','payments','subscriptions','notifications','restaurants')
ORDER BY idx_scan DESC;
```

---

2. Application Cache

A unified cache layer (`lib/cache/index.ts`) was introduced with two drivers:

Driver	When active	Package
Redis	`REDIS_URL` env is set AND `ioredis` is installed	`ioredis` (optional)
In-memory LRU	default fallback — always available	`lru-cache` (installed)

The cache is a `globalThis` singleton so it survives Next.js hot-reload in dev.

Usage

```
import { cache, cached, invalidate, cacheKey, CACHE_TTL } from '@/lib/cache';

// Direct get/set
await cache.set('metrics:revenue', data, CACHE_TTL.MEDIUM); // 5 min ttl
const data = await cache.get<Metrics>('metrics:revenue');

// Read-through helper
const result = await cached(
  cacheKey('platform', 'revenue', '30d'),
  CACHE_TTL.SHORT,
  async () => computeRevenueMetrics()
);

// Invalidation (exact or glob)
await invalidate('metrics:revenue');
await invalidate('platform:*');
```

Available TTL presets

- `CACHE_TTL.TINY` — 10 s (hot metrics)
- `CACHE_TTL.SHORT` — 1 min
- `CACHE_TTL.MEDIUM` — 5 min
- `CACHE_TTL.LONG` — 1 h
- `CACHE_TTL.XLONG` — 24 h

Enabling Redis in Production

1. Provision a Redis instance (Upstash, AWS ElastiCache, Redis Cloud, etc.)
2. Add to `.env`:
   ```
   env
   REDIS_URL=redis://default:<password>@<host>:<port>
   ```
3. Install ioredis:
   ```
   bash
   cd nextjs_space && yarn add ioredis
   ```
4. Restart the app. You'll see `[cache] Using Redis driver` on startup.

If the Redis URL is set but `ioredis` isn't installed, the app logs a warning and continues with the in-memory driver (no crash).

Cache admin endpoint

`GET /api/admin/cache` — returns driver stats (requires OWNER or ADMIN)

`DELETE /api/admin/cache` — flush entire cache
`DELETE /api/admin/cache?pattern=platform:*` — flush matching keys

---

3. Platform Admin Dashboard

Available at `/admin/platform` (OWNER or ADMIN only). Displays:

- **Receita (Revenue)**
- MRR approximation from active subscriptions
- Revenue last 30 days / last 7 days
- Revenue by gateway (Stripe vs MP)
- **Churn**
- Cancelled subscriptions last 30 days
- Net churn rate
- Trial → paid conversion rate
- **Issues / Health**
- Failed payments last 24 h
- Disputes / chargebacks open
- Critical notifications
- Restaurants in trial expiring in 7 days
- **Customers overview**
- Total active / paused / cancelled
- Top 10 customers by revenue

All aggregations use the new indexes + cache layer (5 min TTL).

---

4. Customer Management Portal

Available at `/admin/customers` (OWNER or ADMIN only). Allows:

- **List & search** restaurants with filters (status, tier, subscription status)
- **View details** per customer: subscription history, payments, users, usage
- **Actions**:
- Suspend / reactivate account
- Change subscription tier manually
- Extend trial
- Impersonate owner (for support)

API endpoints:
- `GET /api/admin/customers` — list with filters
- `GET /api/admin/customers/[id]` — details
- `PATCH /api/admin/customers/[id]` — update (suspend, tier, trial)
- `GET /api/admin/customers/[id]/metrics` — usage & revenue metrics